

app details

▼ Features

Let's lock this into a feature set that feels realistic, useful, and technically strong all at once. We want features that connect naturally so the app feels like one coherent system, not five separate ideas.

For this app, the strongest choice is to build it as a **last-mile delivery dispatch and tracking app** with **3 roles**:

- customer
- delivery agent
- admin

And the best **5 features** would be:

1. Delivery request booking

Customer can:

- create a pickup/delivery request
- enter address, package details, priority, and preferred time slot
- view request summary before confirming

Why keep it:

- this is the starting point of the whole system
- without this, there is nothing to schedule or track

Concepts covered:

- OOP models
- forms
- validation
- CRUD
- local/database storage

2. Smart slot assignment and conflict-free scheduling

System automatically:

- checks available delivery windows
- prevents overlapping assignments
- assigns the best feasible slot
- reschedules when a slot is unavailable

Why keep it:

- this is the most technical and impressive feature
- this is where your algorithms and dynamic programming can come in

Concepts covered:

- interval scheduling
- greedy algorithms
- dynamic programming
- constraint checking
- optimization

3. Delivery agent assignment and route optimization

System/admin can:

- assign orders to agents
- group deliveries efficiently
- generate an optimized order of stops
- reduce travel time and workload imbalance

Why keep it:

- this makes the app feel like a real operations product
- it is much more impressive than just "track delivery"

Concepts covered:

- graphs
- shortest path / routing logic
- sorting
- greedy logic
- priority queues

4. Live delivery status tracking

Customer and admin can see:

- order placed
- scheduled
- assigned to agent
- out for delivery
- delivered / failed / rescheduled

Agent can update status at each stage.

Why keep it:

- this is essential for realism
- it connects all the roles together

Concepts covered:

- state management
- database updates
- RecyclerView/UI refresh
- notifications
- event-driven flow

5. Admin/agent dashboard

Admin or agent can:

- view all active deliveries
- filter by pending/completed/delayed
- view today's assigned route

- monitor workload and delivery progress

Why keep it:

- this gives the app a proper management side
- it makes the project look more complete and professional

Concepts covered:

- role-based access
- search/filter/sort
- dashboards
- authentication
- clean architecture

This is the best final feature set

So I'd recommend:

- **Delivery request booking**
- **Smart slot assignment and scheduling**
- **Delivery agent assignment + route optimization**
- **Live status tracking**
- **Admin/agent dashboard**

Why this set is the strongest

Because the flow makes sense:

- customer places request
- system schedules it
- system assigns agent and route
- agent updates progress
- admin monitors everything

So every feature supports the next one.

If your teacher wants only 4 features

Then keep these 4:

- Delivery request booking
- Smart scheduling
- Agent assignment + route optimization
- Live status tracking

That is the most powerful 4-feature version.

Best title for the project

Smart Delivery Dispatch and Tracking App

or

Last-Mile Delivery Scheduling and Tracking System

Best concept coverage from this

This one app can honestly include:

- Java
- OOP
- web app development
- database
- authentication
- role-based system
- algorithms
- dynamic programming
- graph logic
- scheduling
- optimization
- notifications
- clean architecture

Next I can map these 5 features to exact technical concepts one by one so you can explain the project properly in viva and on your resume.

▼ Target Users

The target users can be **three groups**, which actually makes the app stronger:

1. Customers

These are people who:

- book a pickup or delivery
- choose time slots
- track order status
- get notifications

They use the app to request the service.

2. Delivery agents

These are the people who:

- receive assigned deliveries
- view optimized routes
- update order status
- manage completed and pending tasks

They use the app to carry out the delivery work.

3. Admin / business operator

This is the company side, who:

- manages all orders
- monitors agents
- checks delays and workload
- handles scheduling and system control

They use the app to run operations.

So the app is best described as a **multi-user delivery management system**.

If you want one primary target user for your project description, the best answer is:

Primary target user: small delivery businesses and their customers

That sounds practical because the app helps:

- customers place requests
- businesses schedule deliveries efficiently
- agents complete deliveries faster

So on paper, the target audience could be written as:

"The app is designed for small and medium delivery service providers, their delivery personnel, and customers who need parcel pickup and delivery services."

That is probably the strongest way to present it.

If you want, I can also help you choose whether this should be framed as a **B2C app**, **business operations app**, or **multi-role system** for maximum resume value.

▼ Like Swiggy/Zomato

Yes — **it could be that kind of system**, but **only the delivery/logistics side**, not the full food-ordering platform.

So think of it like:

- **not** "Swiggy/Zomato full app"
- but **the dispatch, assignment, scheduling, and tracking engine behind it**

That is actually a **better student project scope**, because building full Swiggy is too broad.

Your app could focus on:

- order/pickup request creation
- delivery agent assignment
- time-slot allocation
- route optimization

- live delivery status tracking
- admin monitoring dashboard

So yes, the app can absolutely be framed as:

"a delivery tracking and dispatch management app inspired by the logistics workflow of apps like Zomato or Swiggy."

What it would exclude

You would likely skip:

- restaurant browsing
- menu management
- cart
- payments
- coupons
- ratings/reviews

Because those belong to the commerce side, not the delivery optimization side.

What it would include

You would focus on:

- how deliveries are assigned
- how slots are managed
- how agents are tracked
- how routes are optimized
- how status updates happen

That makes the project more technical, because it shifts attention to:

- algorithms
- scheduling
- conflict resolution
- route logic

- dynamic programming
- role-based workflows

Best way to describe it

You could present it as:

"A Java-based mobile delivery dispatch and tracking app that manages order assignment, conflict-free slot scheduling, route optimization, and live status updates for delivery operations."

That sounds much stronger than just saying:

"an app like Swiggy."

Because "like Swiggy" sounds too huge and vague, while this sounds specific and technically serious.

Even better framing

Instead of saying "Swiggy clone," say:

- **delivery dispatch and tracking system**
- **last-mile delivery management app**
- **smart pickup and delivery scheduler**
- **delivery operations optimization app**

Those sound more professional on a resume.

In one line

Yes — it can be like the **delivery tracking and dispatch side of Zomato/Swiggy**, without the restaurant-ordering side.

The strongest version is:

Swiggy/Zomato logistics layer, not Swiggy/Zomato full product.

I can help you decide the exact 4–5 features next so it feels clearly scoped and impressive.

▼ 3 Week Implementation Plan

Yes — here's the **remade 3-week implementation plan** for the **web app** version, with the **5th feature kept as a real shared feature** and its own

branch.

Project

Last-Mile Delivery Scheduling and Tracking Web App

Team split

Tanushree

- Delivery Booking
- Smart Slot Assignment and Conflict-Free Scheduling

Bhavya

- Live Tracking
- Admin/Agent Dashboard

Shared by both

- Delivery Agent Assignment + Route Optimization
 - Integration
 - Testing
 - Bug fixing
 - Documentation
 - Final demo prep
-

GitHub branch structure

- `main` → final stable version
- `dev` → combined testing branch
- `feature-delivery-booking`
- `feature-slot-scheduling`
- `feature-live-tracking`
- `feature-dashboard`
- `feature-agent-assignment-route`

Branch flow

Each feature is built in its own branch, then merged into `dev`, tested there, and only after stability is confirmed, merged into `main`.

So the flow is:

feature branch → **dev** → **main**

3-Week Implementation Plan

Week 1: Setup + core feature foundations

Goal

Set up the project properly and build the first working versions of the independent features.

Day 1 — Project setup and planning

Tanushree + Bhavya

- finalize feature list and role flows:
 - customer
 - delivery agent
 - admin
- finalize pages/screens for the web app
- decide backend structure, frontend structure, and database schema
- create GitHub repository
- create all branches
- write README first draft
- decide coding conventions, folder structure, and naming rules

Deliverables

- repo initialized
- branch structure ready

- project folder structure created
- base README created

Commits

- `initialized project structure`
 - `created branch workflow and README draft`
-

Day 2 — Base architecture and schema

Tanushree

- create backend models for:
 - DeliveryRequest
 - TimeSlot
 - Booking
- start Delivery Booking frontend page

Bhavya

- create backend models for:
 - DeliveryStatus
 - TrackingEvent
 - DashboardSummary
- start Live Tracking and Dashboard frontend skeletons

Deliverables

- model classes ready
- initial frontend layouts started

Commits

- Tanushree: `added booking and slot model classes`
 - Bhavya: `added tracking and dashboard model classes`
-

Day 3 — Delivery Booking and Live Tracking UI

Tanushree — feature-delivery-booking

- build booking form page
- add fields:
 - pickup location
 - drop location
 - package details
 - priority
 - preferred time slot
- create confirmation page

Bhavya — feature-live-tracking

- build tracking page UI
- create visual status timeline:
 - placed
 - scheduled
 - assigned
 - out for delivery
 - delivered
 - failed/rescheduled

Deliverables

- first version of booking UI
 - first version of tracking UI
-

Day 4 — Backend logic for first features

Tanushree

- implement booking form validation
- connect booking form to database insert logic

Bhavya

- implement delivery status transition logic
- define allowed status update sequence

Deliverables

- booking submission working partially
- tracking logic working partially

Commits

- Tanushree: `implemented booking validation and save logic`
 - Bhavya: `implemented delivery status transition flow`
-

Day 5 — Scheduling and Dashboard begin

Tanushree — feature-slot-scheduling

- define scheduling rules
- create overlap detection methods
- build slot result/suggestion page

Bhavya — feature-dashboard

- build dashboard page
- add sections for:
 - pending deliveries
 - completed deliveries
 - delayed deliveries
 - assigned deliveries

Deliverables

- scheduling module started
 - dashboard UI started
-

Day 6 — First feature refinement

Tanushree

- improve booking CRUD flow
- allow request creation and viewing

Bhavya

- improve tracking UI
- connect tracking page to backend data source

Deliverables

- booking feature nearly functional
 - tracking feature nearly functional
-

Day 7 — First merge into dev

Tanushree + Bhavya

- review each other's branches
- merge stable code into `dev`
- test whether booking, tracking, and base pages coexist correctly
- fix merge issues

Deliverables

- integrated Week 1 progress in `dev`

Commits

- `merged booking feature into dev`
 - `merged tracking feature into dev`
 - `fixed integration issues after first merge`
-

Week 2: Complete individual features

Goal

Finish all four owned features end-to-end.

Day 8 — Complete Delivery Booking and Dashboard backend

Tanushree — feature-delivery-booking

- complete create/view/edit/delete flow
- ensure request data saves properly
- improve confirmation and error handling

Bhavya — feature-dashboard

- implement backend data retrieval for dashboard
- populate delivery counts and grouped sections

Deliverables

- delivery booking fully functional
 - dashboard data flow started
-

Day 9 — Scheduling logic and Dashboard filters

Tanushree — feature-slot-scheduling

- implement slot availability checks
- assign feasible slot to each new booking
- suggest alternative slot if preferred one is unavailable

Bhavya — feature-dashboard

- add filter and sort logic:
 - pending
 - completed
 - delayed
 - assigned
- connect filters to data

Deliverables

- scheduling logic functional

- dashboard filtering functional
-

Day 10 — Technical depth improvement

Tanushree

- strengthen scheduling logic using:
 - interval overlap checking
 - greedy logic
 - DP-style best slot selection if needed
- test cases:
 - no free slots
 - overlapping requests
 - invalid time windows

Bhavya

- improve tracking logic:
 - invalid transitions blocked
 - delayed/failed states supported
- improve dashboard counts and status grouping

Deliverables

- smart scheduling technically strong
 - tracking/dashboard more realistic
-

Day 11 — Internal feature integration

Tanushree

- connect Delivery Booking to Slot Assignment
- booking submission should automatically trigger slot validation

Bhavya

- connect Live Tracking to Dashboard
- dashboard should reflect updated delivery statuses

Deliverables

- Tanushree's two features connected
 - Bhavya's two features connected
-

Day 12 — Individual testing and fixes

Tanushree

- fully test booking + scheduling flow
- fix UI/backend mismatches

Bhavya

- fully test tracking + dashboard flow
- fix UI/backend mismatches

Deliverables

- four independent features complete
-

Day 13 — Merge all four features into dev

Tanushree + Bhavya

- merge:
 - `feature-delivery-booking`
 - `feature-slot-scheduling`
 - `feature-live-tracking`
 - `feature-dashboard`into `dev`
- test the full system so far

Deliverables

- `dev` contains all four main owned features
-

Day 14 — Stabilization day

Tanushree + Bhavya

- clean code
- refactor repeated logic
- improve naming
- check database consistency
- prepare for shared feature development

Deliverables

- stable base app ready for 5th feature
-

Week 3: Shared 5th feature + final integration

Goal

Build the shared feature, integrate everything, and prepare the final submission.

Day 15 — Start shared feature branch

Both on `feature-agent-assignment-route`

Tanushree

- build frontend for agent assignment view
- create page showing assigned deliveries and route order

Bhavya

- implement backend logic for agent assignment
- assign deliveries based on availability/load

Deliverables

- agent assignment UI started
 - backend assignment logic started
-

Day 16 — Route optimization logic

Tanushree

- improve shared feature frontend
- add route display table/card flow for assigned orders

Bhavya

- implement route optimization logic
- determine best sequence of deliveries
- handle simple shortest/nearest-next style logic

Deliverables

- route logic functional
 - shared feature UI progressing
-

Day 17 — Shared feature integration

Tanushree + Bhavya

- connect assignment and route logic to frontend
- ensure assigned deliveries appear correctly for agent/admin
- test with multiple deliveries and multiple agents

Deliverables

- shared 5th feature working
-

Day 18 — Merge 5th feature into dev

Tanushree + Bhavya

- merge `feature-agent-assignment-route` into `dev`
- test complete end-to-end flow:
 - create booking
 - assign slot
 - assign delivery agent
 - optimize route

- update tracking
- reflect in dashboard

Deliverables

- full feature set combined in `dev`
-

Day 19 — Full system testing

Tanushree + Bhavya

Test:

- duplicate bookings
- no available slots
- invalid status updates
- no available agents
- delayed delivery cases
- dashboard data mismatches
- broken navigation or DB issues

Deliverables

- major bugs identified and fixed
-

Day 20 — Polish and documentation

Tanushree

- polish booking and scheduling pages
- improve form clarity, messages, and slot suggestion display

Bhavya

- polish tracking and dashboard pages
- improve layout, filters, and status readability

Both

- finalize README:

- objective
- features
- tech stack
- OOP concepts used
- algorithms used
- team contribution
- setup instructions

Deliverables

- polished app
 - proper documentation
-

Day 21 — Final merge and demo prep

Tanushree + Bhavya

- final test on `dev`
- merge stable code into `main`
- prepare demo flow
- prepare viva explanation for:
 - Tanushree's two features
 - Bhavya's two features
 - shared 5th feature
- create screenshots / submission assets

Deliverables

- submission-ready `main`
 - demo-ready project
-

Final ownership summary

Tanushree

Feature 1: Delivery Booking

Frontend:

- booking form
- booking confirmation

Backend:

- booking validation
- CRUD
- DB integration

Feature 2: Smart Slot Assignment

Frontend:

- slot result page
- alternative slot suggestion page

Backend:

- overlap detection
 - conflict-free scheduling
 - slot assignment logic
 - optimization logic
-

Bhavya

Feature 3: Live Tracking

Frontend:

- tracking page
- status timeline

Backend:

- delivery status transitions
- DB updates for status

Feature 4: Dashboard

Frontend:

- admin/agent dashboard
- filter/sort controls

Backend:

- grouped delivery data
 - filter/sort logic
 - summary counts
-

Shared

Feature 5: Delivery Agent Assignment + Route Optimization

Frontend:

- assignment view
- route display

Backend:

- agent allocation
 - route ordering logic
 - load balancing basics
-

Commit strategy

Each of you should push **small, regular commits**, not giant ones.

Good commit examples for Tanushree

- `created delivery booking form page`
- `added validation for booking form`
- `implemented CRUD for delivery requests`
- `added slot overlap detection`
- `implemented alternative slot suggestion`

Good commit examples for Bhavya

- `created delivery tracking timeline`
- `implemented delivery status transition rules`
- `built dashboard cards and tables`
- `added filter and sorting logic`
- `connected dashboard to live delivery data`

Good shared commits

- `added agent assignment backend logic`
 - `implemented route ordering for assigned deliveries`
 - `integrated assignment feature with dashboard and tracking`
 - `fixed end to end delivery flow bugs`
-

Best practical note

For the shared branch, both of you can either:

- commit directly to `feature-agent-assignment-route`, or
- make temporary sub-branches and merge into it.

For a student project, direct commits to that feature branch are fine as long as you both pull regularly and communicate.

If you want, I can turn this into a **clean table format** with columns: **Day | Tanushree | Bhavya | Branch | Deliverable.**